

Läuft der DATEV MCP-Server vollständig auf Azure

ZUORDNUNG DER KOMPONENTEN *was aus jedem Vercel-Baustein wird*

HEUTE	AUF AZURE	ANMERKUNG	AUFWAND
Vercel Serverless Functions	Azure Container Apps alternativ App Service for Containers	Verträgt Streaming und lange Verbindungen deutlich besser als ein Modell mit einer Funktion pro Request – genau das braucht der MCP-Transport in <code>app/api/[transport]/route.ts</code> .	ÜBERNEHMEN
Upstash Redis (REST)	Azure Managed Redis oder Azure Cache for Redis	Die einzige echte Codeänderung. <code>@upstash/redis</code> spricht REST, Azure spricht RESP – der Client wird auf <code>ioredis</code> umgestellt. Alle Aufrufe laufen ohnehin schon durch <code>safeRedis()</code> .	ERSETZEN
Vercel Umgebungsvariablen	Azure Key Vault mit Managed Identity	<code>DATEV_CLIENT_SECRET</code> und <code>MCP_AUTH_TOKEN</code> kommen beim Start aus dem Key Vault. In der Container-Definition steht dann kein Geheimnis mehr.	ÜBERNEHMEN
Vercel Edge und Domain	Azure Front Door mit WAF, dazu Azure DNS	TLS, Web Application Firewall und die produktive Wunschdomain – die DATEV für die Redirect-URL ohnehin verlangt (siehe unten).	ÜBERNEHMEN
Deploy über Git	Azure Container Registry GitHub Actions oder Azure DevOps	Normaler Container-Build. Dafür fehlen im Repository bislang ein Dockerfile und die Standalone-Ausgabe von Next.js.	ERGÄNZEN
Gemeinsames Bearer-Token	API Management mit Microsoft Entra ID	Optional, aber der Punkt, an dem „Enterprise“ konkret wird: Identität pro Aufrufer, Widerruf, Drosselung und ein Audit-Trail statt eines geteilten Geheimnisses.	AUSBAUEN

ZIELARCHITEKTUR *eine Region, Germany West Central oder West Europe*

AUFRUFER	→ RAND	→ RECHENLEISTUNG	→ ZUSTAND	→ GEGENSTELLE
nuwacom-Agent MCP-Client	Front Door + WAF APIM · Entra ID	Container Apps Next.js · <code>/api/mcp</code>	Redis (Private Endpoint) Key Vault	*.api.datev.de login.datev.de

Abschottung. Die Container Apps liegen in einem eigenen VNet, Redis und Key Vault sind nur über Private Endpoints erreichbar, und ausgehend sind allein die beiden DATEV-Hosts freigegeben.

Datenhaltung. Rechenleistung, Cache und Geheimnisse liegen in einer EU-Region, sinnvollerweise Germany West Central oder West Europe. Bei Belegdaten deutscher Kanzleien ist das der deutlichste Vorteil gegenüber der heutigen Plattform.

Betrieb. Mit `minReplicas: 1` entfällt der Kaltstart, der heute bei jedem Aufwachen einen zusätzlichen Weg zu Redis erzwingt.

WAS DER UMZUG BRINGT

Datenresidenz

Jede Komponente in einer EU-Region, in Ihrem eigenen Abonnement und Ihrem eigenen Tenant.

Netzwerk

VNet-Integration, Private Endpoints und eine Ausgangs-Freigabeliste. Nichts davon lässt sich auf der heutigen Plattform abbilden.

Identität und Nachweis

Entra ID am Gateway, Managed Identity zu Key Vault und Redis, Azure Monitor für Aufrufe und Token-Erneuerungen.

WAS AZURE NICHT LÖST

Ein Token-Satz für alle

Die Token liegen unter einem einzigen Schlüssel, `datev:tokens`. Ein DATEV-Konto bedient also die gesamte Installation. Mandantentrennung bräuchte Schlüssel je Mandant und einen eigenen Verbindungsablauf – das ist Anwendungsarbeit, keine Infrastrukturfrage.

Refresh-Token nach rund elf Stunden

Jemand muss ungefähr einmal pro Arbeitstag `/api/datev/connect` im Browser bestätigen. Die Variante mit `offline_access` hält zwei Jahre, bindet aber an einen einzelnen Mandanten und verliert `list_clients`. Das gibt DATEV vor, daran ändert Azure nichts.

Namensregel für die Redirect-URL

DATEV lehnt jede Redirect-URL ab, in deren *Domain* „datev“ vorkommt – im Pfad ist es erlaubt. Produktiv muss die Domain zudem eindeutig auflösbar sein. Der Host heißt also `mcp.exelentic.com`, nicht `datev-mcp....`

AM CODE ZU ÄNDERN

`lib/redis.ts`

Upstash-REST-Client gegen einen RESP-Client tauschen. Gekapselt hinter `safeRedis()`, kein anderes Modul ist betroffen.

`next.config.js` + `Dockerfile`

Standalone-Ausgabe aktivieren und einen Container-Build ergänzen. Neue Datei, bestehende Logik bleibt unberührt.

`app/api/[transport]/route.ts`

`maxDuration = 60` ist eine Vercel-Anweisung und läuft künftig ins Leere. Container Apps kennt keine vergleichbare Obergrenze pro Request – für lange Tool-Aufrufe ist das ein Gewinn.